

Mobile Devices and Applications

UNIT 3 – APP LIFECYCLE AND UI

Summary



App Components



XML Layouts



Procedural Interface



Containers

App Components

The basic components of an Android App

Activities

- *android.app.Activity* – base class

Services

- *android.app.Service* – base class

Content providers

- *android.content.ContentProvider* – base class

Message receiver

- *android.content.BroadcastReceiver* – base class
- *messages*

Intents

- *android.content.Intent* – base class

Activities

A single, standalone module of app functionality

Fully reusable and interchangeable building blocks

Associated with application's windows

Just a single main activity

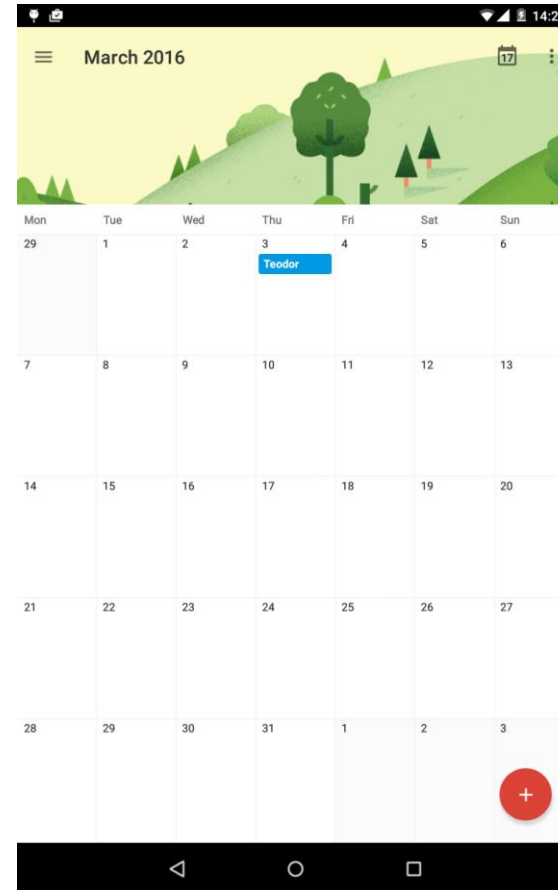
Works with visual controls

- Inherited from the View class

For passing params one can use Intents

For returning results they must be started as sub-activities

Activities



Activities

Have their own life cycle managed by the ART

- Multiple stages of creation and destruction
 - With the possibility of saving the activity's intermediate states
 - onSaveInstanceState – called before an activity must be destroyed
 - onRestoreInstanceState – called if a previous state of an activity was saved

Two different states from the activity's lifecycle perspective

- Persistent state (database, content providers or files)
- Dynamic state (UI state or instance state)

Application class

Has access to applications' methods and settings

Base class for maintaining global application state

Is the first class instantiated from the app's package

Has a specific context

- Available on the entire life cycle of an application
- **getApplicationContext()**

Context interface

An abstract class defined in the package *android.content*

Gives access to the application's environment

Used for accessing application-specific resources

Used for launching new activities

Has access to system services

Has access to internal files and databases

UI Android apps

Include one or more activities

Each activity can have a visual component associated with it

- Code defined
- Procedural defined in XML files

Include resources:

- Associated with the UI
- Files processed inside the app

Activities

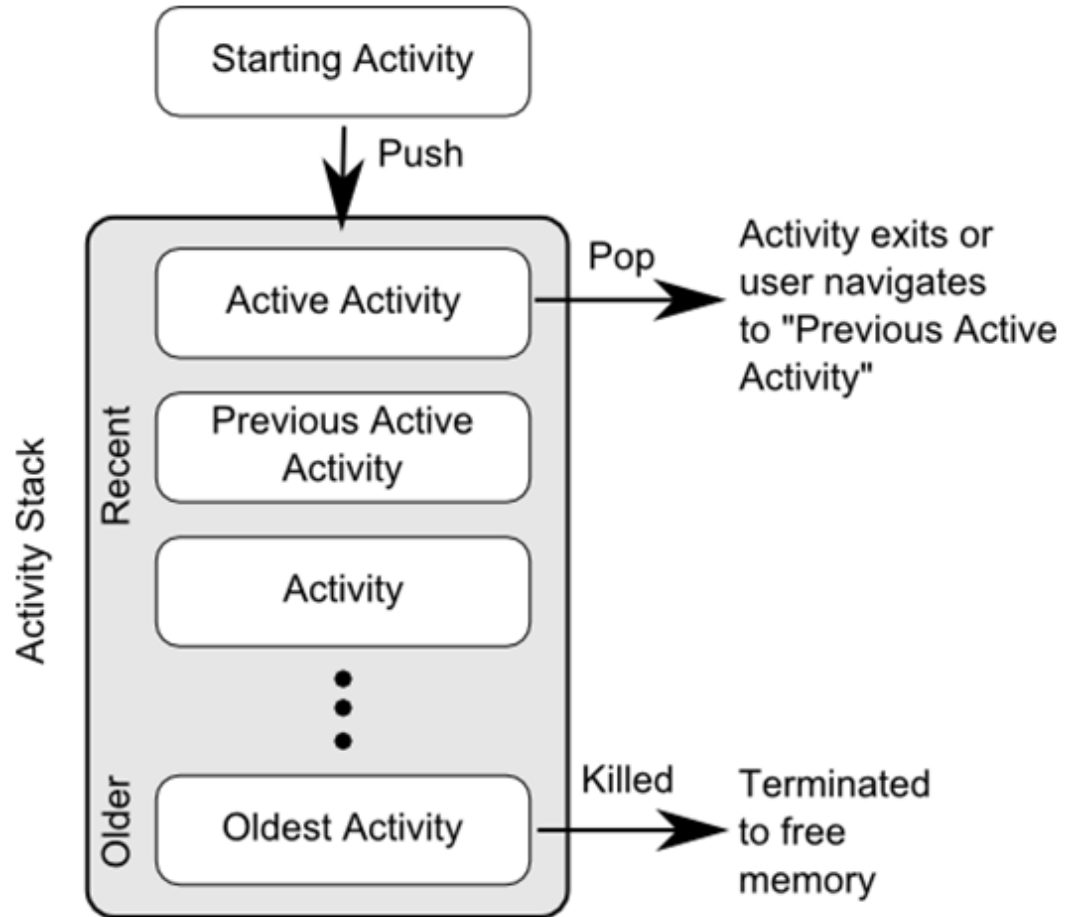
Each activity is inserted into a stack of activities managed at the application level by the ART

- The first activity is referred to as the active

An activity has four activity states

- Active / running
- Paused
- Stopped
- Killed

Activities



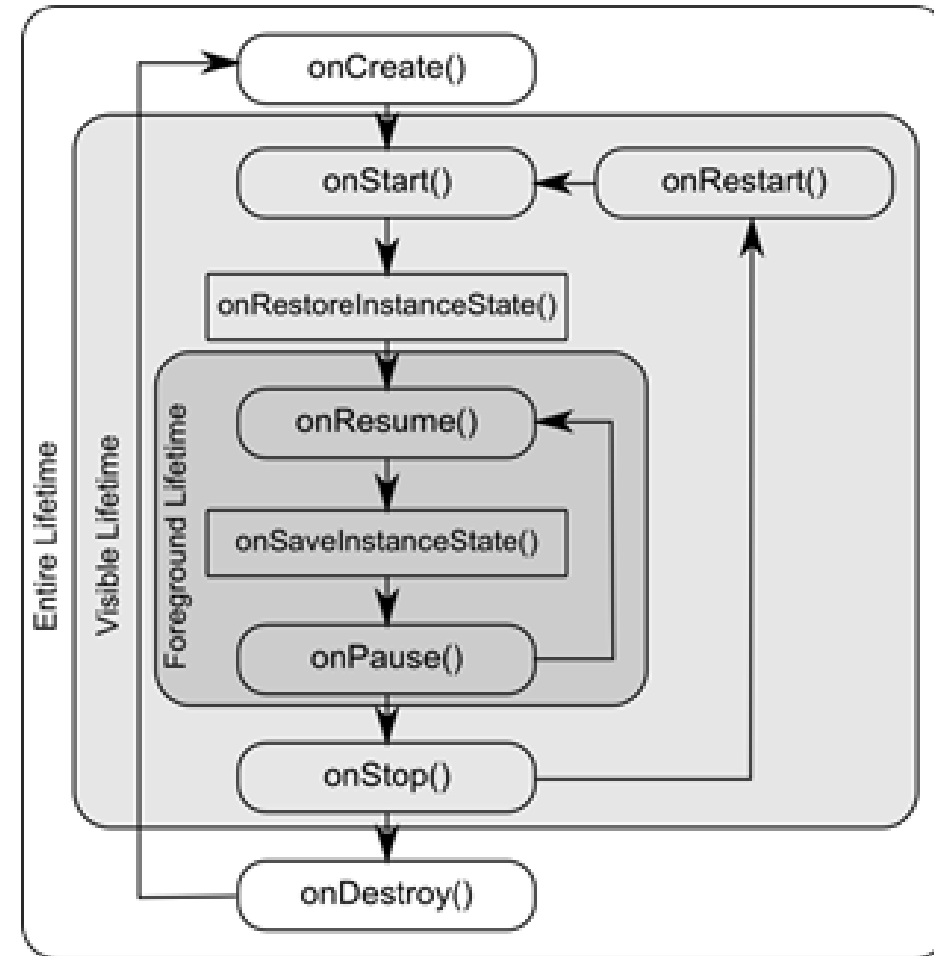
Activity lifetimes

Entire lifetime – everything that happens between the initial call to `onCreate()` and the `onDestroy()`

Visible lifetime – from `onStart()` up to `onStop()`

Foreground lifetime – between the calls to `onResume()` and `onPause()`

Activity lifetimes



Lightweight Activity

```
import android.app.Activity;
import android.os.Bundle;

public class TestActivity extends Activity
{
    @Override
    public void onCreate(Bundle stare) {
        super.onCreate(stare);
        setContentView(view);
    }
}
```

Configuration changes

Changing device orientation

- Rotations / IO events / alarms
- Showing or hiding virtual keyboard

Changing language

- Regional settings

Configuration changes

```
<activity android:name=".MyActivity"  
    android:configChanges="orientation|keyboardHidden"  
    android:label="@string/app_name">
```

Configuration changes

```
@Override
public void onConfigurationChanged(Configuration newConfig) {
    super.onConfigurationChanged(newConfig);

    // Checks the orientation of the screen
    if (newConfig.orientation == Configuration.ORIENTATION_LANDSCAPE) {
        Toast.makeText(this, "landscape", Toast.LENGTH_SHORT).show();
    } else if (newConfig.orientation == Configuration.ORIENTATION_PORTRAIT){
        Toast.makeText(this, "portrait", Toast.LENGTH_SHORT).show();
    }
}
```

Intents

Intent objects

Asynchronous messages

Utility

- **Invoking activities**
- **Calling services**
- **Sending messages**
- **Sharing data**

Intents

Mechanism by which one activity can launch another one

Two types of Intents

- Explicit – they request the launch of a specific activity by specifying its class name
- Implicit – states the type of action to be performed or providing data of a specific type on which the action is to be performed

Broadcast Intents – a system intent sent out to all the applications that have registered an interested Broadcast Receiver

- Normal (asynchronous) – sent to all interested Broadcast Receivers
- Ordered – sent to one receiver which process it and decides either to pass it or to block it

Intent characteristics

Action

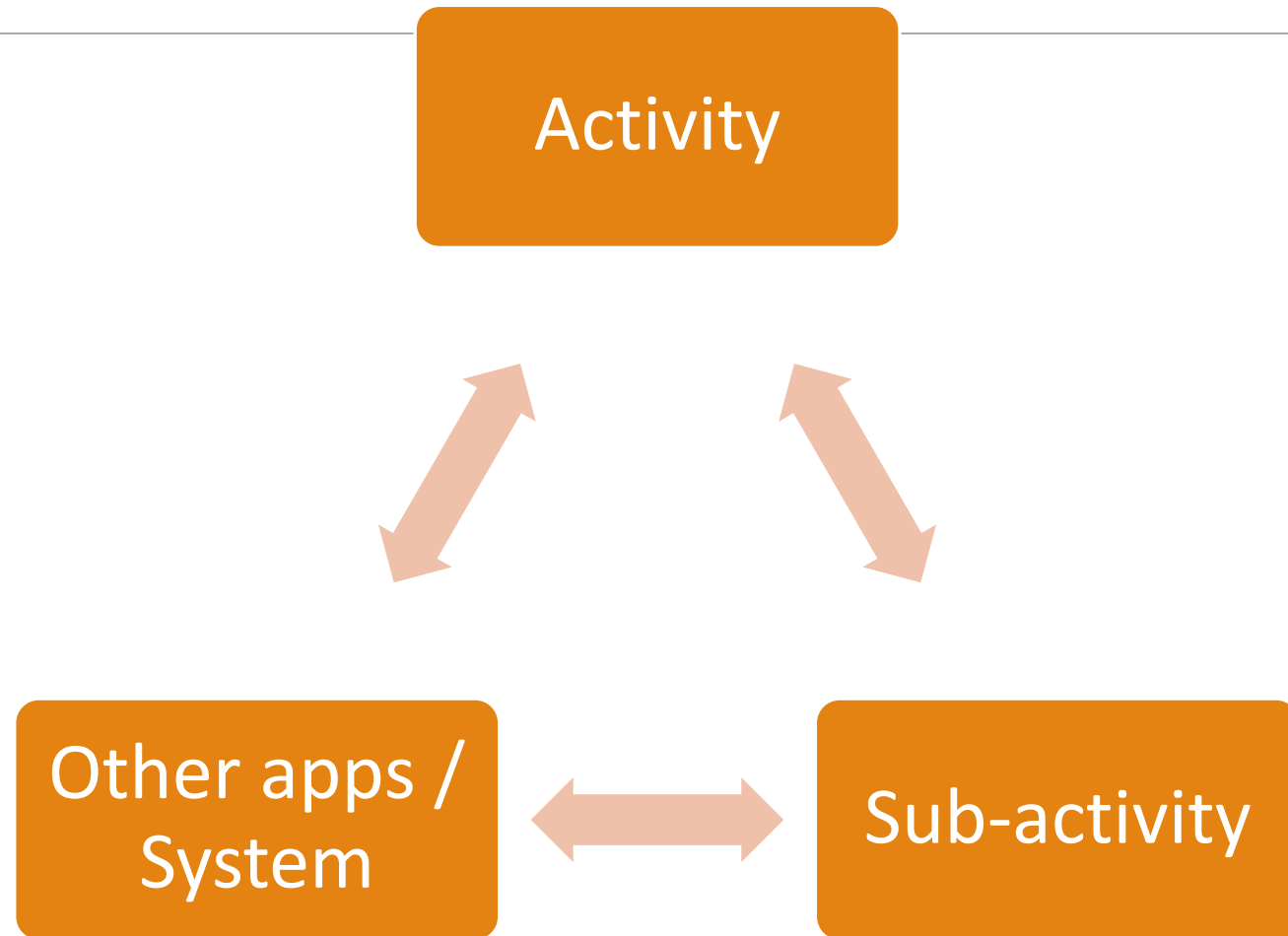
Data

Category

Additional data

Destination

App communications



App communications

Activity => Other activity

- Send: intent (startActivity)
- Receive: Bundle

Activity ⇔ Sub-activity

- Send: intent (startActivityForResult)
- Receive: Bundle

Call a second activity

Explicit intent

Parameters:

- The app's context
- Second activity's class name

Methods (Context class)

- **startActivity(activityIntent)**

Sending additional data

Attached bundle object

- Data container

Adding data

- **putExtra**(key, value)

Adding a whole container

- putExtras(bundle)

Getting data

- getFloatExtra(), getStringExtra()

Getting an entire container

- Bundle **getExtras**()

App communications

Activity => Other apps' activities

- Intent-filter action
- Intent-filter category
- Intent-filter data

System or other apps => Activity

- Using intent-filters registered to broadcasted intents

Intent resolutions

<Action>

- Declares the intent action accepted, in the name attribute. The value must be the literal string value of an action, not the class constant.

[Data]

- Declares the type of data accepted, using one or more attributes that specify various aspects of the data URI (scheme, host, port, path) and MIME type.

[Category]

- Declares the intent category accepted, in the name attribute. The value must be the literal string value of an action, not the class constant.

AndroidManifest.xml

<activity

android:name=".MainActivity"

android:label="@string/app_name" >

<!-- This activity is the main entry, should appear in default activity launcher -->

<intent-filter>

<action android:name="android.intent.action.MAIN" />

<category android:name="android.intent.category.LAUNCHER" />

</intent-filter>

<!-- This activity handles "SEND" actions with text data -->

<intent-filter>

<action android:name="android.intent.action.SEND"/>

<category android:name="android.intent.category.DEFAULT"/>

<data android:mimeType="text/plain"/>

</intent-filter>

</activity>

Implicit Intents

```
//1.// - intent for web browser  
Intent callIntent = new Intent(Intent.ACTION_VIEW,  
    Uri.parse("http://www.google.com"));
```

Implicit Intents

//2.// - intent for sending text to another app

```
Intent callIntent = new Intent(Intent.ACTION_SEND);  
callIntent.putExtra(Intent.EXTRA_TEXT, "Hello from an Android  
app");  
callIntent.setType("text/plain");
```

Implicit Intents

//3.// - intent for calling a phone number

```
Intent callIntent = new Intent(Intent.ACTION_DIAL, Uri.parse("tel:0740111222"));
```

Implicit Intents

//4.// - intent for starting camera capture

```
Intent callIntent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);
```

Intent receiver

```
void onCreate (Bundle savedInstanceState) {  
    // Get intent, action and MIME type  
    Intent intent = getIntent();  
    String action = intent.getAction();  
    String type = intent.getType();  
  
    if (Intent.ACTION_SEND.equals(action) && type != null) {  
        if ("text/plain".equals(type)) {  
            handleSendText(intent); // Handle text being sent  
        } else if (type.startsWith("image/")) {  
            handleSendImage(intent); // Handle single image being sent  
        }  
    }  
}
```

Launching an activity

```
/*
```

From the current activity a new activity called TestActivity is launched

```
*/
```

```
Intent intent = new Intent(this, TestActivity.class);
```

```
//adding parameters – test id
```

```
intent.putExtra("idTest", 1001);
```

```
//launching the intent
```

```
startActivity(intent);
```

Getting data

@Override

```
public void onCreate(Bundle stare)
```

```
{
```

```
    super.onCreate(stare);
```

```
    //getting the container
```

```
    Bundle param = getIntent().getExtras();
```

```
    //testing if param is null
```

```
    int idTest = param.getInt("idTest")
```

```
}
```

Resources

C. Boja, C. Ciurea, M. Doinea – Android Mobile Applications: A Practical Development Guide, ASE Press, 2015, 404 pp., ISBN 978-606-34-0033-9

S. Komatineni, D. MacLean – Pro Android 4, Apress, 2012

R. Meier – Professional Android 4 Application Development, Wiley, 2012

P. Pocatilu, Programarea dispozitivelor mobile, Editura ASE, 2012

P. Pocatilu, I. Ivan ș.a. – Programarea aplicațiilor Android, Editura, ASE, 2015

<http://developer.android.com>